DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 3
Manejo de Conectores
SQLite

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

UD3 – SQLite

ÍNDICE

1. SQLite
2. SQLite - Comandos
3. SQLite - Visual
4. SQLite - Java

1. SQLite

SQLite es un sistema de gestión de bases de datos relacional (SGBD-R) compatible con ACID, contenida en una relativamente pequeña biblioteca escrita en C.

SQLite es un **proyecto de dominio público** creado por D. Richard Hipp.

A diferencia de los sistema de gestión de bases de datos cliente-servidor, el motor de SQLite no es un proceso independiente con el que el programa principal se comunica. En lugar de eso, la biblioteca SQLite se enlaza con el programa pasando a ser parte integral del mismo. El programa utiliza la funcionalidad de SQLite a través de llamadas simples a subrutinas y funciones. Esto reduce la latencia en el acceso a la base de datos, debido a que las llamadas a funciones son más eficientes que la comunicación entre procesos.

El conjunto de la base de datos (definiciones, tablas, índices, y los propios datos), son guardados como un solo fichero estándar en la máquina host. Este diseño simple se logra bloqueando todo el fichero de base de datos al principio de cada transacción.

En su versión 3, SQLite permite bases de datos de hasta 2 Terabytes de tamaño, y también permite la inclusión de campos tipo BLOB.

SGBD	**SQLite**
Modelo de Datos	**Relacional**
Tipo de Datos	**Tablas**
Acceso	**Fichero**
Puerto	**NO**
Client Command-Line	**sqlite3**
Client GUI Escritorio (Interfaz Gráfica de Usuario)	**DB Browser for SQLite**
Client GUI Web (Interfaz Gráfica de Usuario)	**NO**

Características

- **Precio:** gratuito
- **Almacenamiento**: un único archivo

2. SQLite - Comandos

El ejecutable para disponer de un intérprete de comandos es **sqlite3.exe**

```
// Abrir intérprete de comandos abriendo una BD
sqlite3 datos/ejemplo.db

// Abrir intérprete de comandos abriendo una BD
// y ejecutando un conjunto de comandos
sqlite3 prueba.db < script.sql
```

El intérprete de comandos dispone de instrucciones propias:

.help	Muestra toda la ayuda
.quit	Sale de SQLite
.show	Muestra el valor actual de algunas variables
.dbconfig	Lista el valor de las opciones
.read file.sql	Ejecuta las instrucciones de un fichero

.cd path	Cambiar de directorio
.shell CMD [Argumentos]	Ejecutar un comando del Sistema Operativo
.shell cls	Limpia la pantalla

.databases	Muestra la bases de datos abierta
.dbinfo	Muestra información del estado de la BD
.open basedatos.db	Abre/Selecciona la base de datos
.tables	Muestra las tablas de la base de datos abierta
.schema tabla	Muestra de una tabla el CREATE TABLE

.headers on|off	Mostrar los nombres de los campos
.mode list|column	Mostrar en formato lista o tabla

Algunos de los comandos SQL que acepta son:

```
CREATE TABLE tbl1 (
      one TEXT,
      two INTEGER,
      PRIMARY KEY (one)
);

INSERT INTO tbl1 (one, two) VALUES ('REG1',31);

SELECT * from tbl1;
```

```
DROP TABLE IF EXISTS departamentos;

CREATE TABLE departamentos (
      dept_no  TINYINT(2) NOT NULL PRIMARY KEY,
      dnombre  VARCHAR(15),
      loc      VARCHAR(15)
);

INSERT INTO departamentos VALUES (10,'CONTABILIDAD','SEVILLA');
INSERT INTO departamentos VALUES (20,'INVESTIGACIÓN','MADRID');
INSERT INTO departamentos VALUES (30,'VENTAS','BARCELONA');
INSERT INTO departamentos VALUES (40,'PRODUCCIÓN','BILBAO');

SELECT dept_no, dnombre, loc FROM departamentos;
```

REPOSITORIO DE DATOS

SQLite dispone de un repositorio de datos, equivalente a INFORMATION_SCHEMA en un MySQL pero mucho más simple.

La tabla **sqlite_master** contiene las tablas e índices de nuestra BD

```
.schema sqlite_master

SELECT type, name, tbl_name, rootpage FROM sqlite_master;

SELECT * FROM sqlite_master;
```

A través de **pragma_table_info** podemos obtener la estructura de una tabla

```
SELECT cid, name, type, [notnull], dflt_value, pk
  FROM pragma_table_info('tabla');
```

cid	Identificador de tipo INTEGER
name	Nombre de la columna
type	Tipo de dato, como por ejemplo INTEGER o TEXT
[notnull]	1 si la columna tiene una restricción NOT NULL, 0 en caso contrario
dflt_value	Valor por defecto NULL si no tiene valor por defecto
Pk	1 si la comlumna es PRIMARY KEY, 0 en caso contrario

Haciendo un cruce entre sqlite_master y **pragma_table_info** podemos obtener los campos de una o todas las tablas de una BD

```
SELECT m.name as table_name, p.name as column_name
FROM   sqlite_master AS m JOIN pragma_table_info(m.name) AS p
WHERE  m.name = 'departamentos'
ORDER BY m.name, p.cid;

SELECT m.name as table_name, p.name as column_name
FROM   sqlite_master AS m JOIN pragma_table_info(m.name) AS p
ORDER BY m.name, p.cid;
```

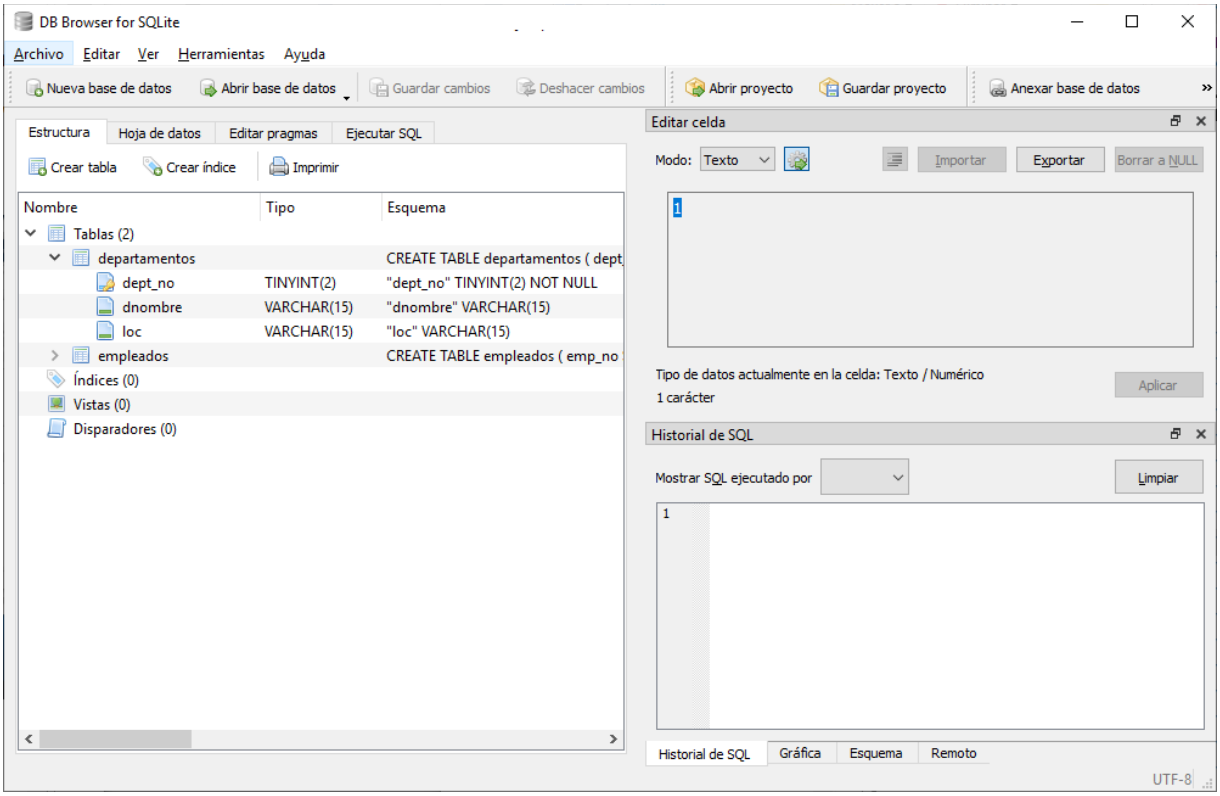
3. SQLite - Visual

Existen varios programas que nos permiten administrar Bases de Datos de SQLite, como **DBeaver**.

También podemos usar "**DB Browser for SQLite**", porque

- no necesita instalación
- es fácil de usar
- es multiidioma
- tiene muchas opciones de configuración y visualización

DB Browser for SQLite – Web Oficial de Oracle
https://sqlitebrowser.org



4. SQLite - Java

El driver JDBC se puede descargar de:

SQLite JDBC - Download
https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc https://jar-download.com/online-maven-download-tool.php

La conexión se realiza con los siguientes parámetros:

```
Class.forName("org.sqlite.JDBC");
conexion = DriverManager.getConnection("jdbc:sqlite:./datos/bddepartamentos.db");
```

Limitaciones

Respecto al resto de SGBD, hay algunas instrucciones en Java que no funcionan. Veamos algunas de ellas:

1) No permite consultas de tipo SCROLL

```
Statement sentencia = conexion.createStatement(
        ResultSet.TYPE_SCROLL_INSENSITIVE,     ResultSet.CONCUR_READ_ONLY
  );
```

Deben ser solo hacia delante:

```
Statement sentencia = conexion.createStatement(
        ResultSet.TYPE_FORWARD_ONLY,     ResultSet.CONCUR_READ_ONLY
  );
```

Esto implica que no podemos ejecutar :

```
resultSet.first();
```

2) No permite conocer los tamaños de los campos a través de MetaData, aunque la instrucción no da error, la información no es correcta. Por ejemplo con la tabla **"libros"**:

```
DatabaseMetaData dbmd = conexion.getMetaData();
ResultSet columnas =
   dbmd.getColumns(null, dbmd.getUserName(), "libros", null);
String tamCol = columnas.getString("COLUMN_SIZE")
```